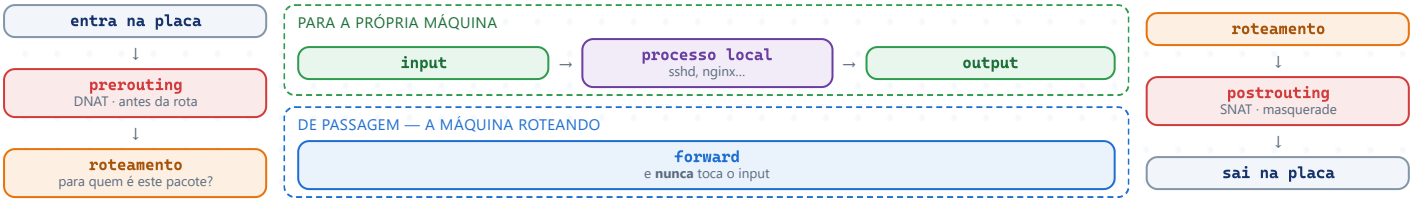
# FIREWALL NO LINUX: NFTABLES E IPTABLES

*O administrador digitou de memória `iptables -A INPUT -p tcp --dport 80 -j ACCEPT` e a porta continuou fechada. A primeira regra da cadeia, posta meses antes por outra pessoa, era um `-j DROP` genérico: a liberação entrou **depois** do descarte, num ponto que o pacote nunca alcançava.*

**083**
GUIAS LINUX

## UM MOTOR (NETFILTER), E O CAMINHO QUE O PACOTE FAZ POR ELE




**A decisão de rota parte o fluxo em dois, e é ela que determina qual gancho o pacote vê** — o erro clássico de quem monta um gateway ou VPN é escrever a regra em `input` quando o pacote está em `forward`. **Um motor, duas interfaces:** o `iptables` (1998) é o que você vai **encontrar** em sistema legado e em tutorial; o `nftables` é o que você vai **escrever** — e no Kali atual o próprio `iptables` já é um invólucro que fala `nftables` por baixo.

## A ORDEM DECIDE · E O ESTADO É O QUE TORNA O FIREWALL USÁVEL

1. `-j DROP` — posta meses antes por outra pessoa — **o pacote para aqui**
2. `-p tcp --dport 80 -j ACCEPT` — acrescentada com **`-A`**, ao fim: **nunca alcançada**

**Lista ordenada, avaliada de cima para baixo, e a primeira que casa decide.** Não há "a regra mais específica vence": há **posição na fila**. Quase todo erro de firewall é erro de ordem, não de sintaxe — e o diagnóstico é uma linha: `iptables -L INPUT -n -v --line-numbers`. A coluna `pkts` é ouro: regra que devia pegar tráfego e mostra **0 packets** não está sendo alcançada.

### ESTADO DA CONEXÃO (CONNTRACK)

| | |
|---|---|
| `new` | o primeiro pacote, tentando abrir conexão — **é só isto que você precisa decidir** |
| `established` | pertence a uma conexão já aceita: **a resposta vem de graça** |
| `related` | nova, mas ligada a outra (erro ICMP, canal de dados do FTP) |
| `invalid` | não casa com estado nenhum — quase sempre lixo, a descartar |

**Sem estado, permitir navegação exigiria abrir todas as portas altas por onde as respostas voltam** — impossível de prever, e abriria a máquina inteira. E `iif lo accept` não é opcional: incontáveis serviços conversam consigo mesmos por `127.0.0.1`.

## NA PRÁTICA: A BASE QUE TODA POLÍTICA SENSATA TEM

kali@kali: /etc/nftables.conf

```
flush ruleset   # após o #!/usr/sbin/nft -f; idempotente
table inet filter {    # inet = IPv4 e IPv6 juntos
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif lo accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept   # NDP é essencial
    tcp dport 22 ct state new limit rate 4/minute accept
    tcp dport { 80, 443 } ct state new accept
  }
  chain forward { type filter hook forward priority 0; policy drop; }
}
$ sudo nft -f /etc/nftables.conf   # troca TUDO, atômico
$ sudo systemctl enable nftables   # e só isto persiste
```

### IPTABLES: ONDE CADA OPÇÃO MORDE

| | |
|---|---|
| `-A` | **A**crescenta ao fim — vai **depois** de um `DROP` existente e nunca é alcançada |
| `-I [n]` | **I**nsere no topo (ou na posição **n**): a forma de pôr uma regra antes das outras |
| `-P` | define a **política padrão** da cadeia |
| `-F` | esvazia a cadeia — com política `DROP`, corta todo o acesso na hora |

### GATEWAY: ENCAMINHAR E MASCARAR

1. `sysctl -w net.ipv4.ip_forward=1`
2. `nft add rule ip nat postrouting oifname "enp0s3" masquerade` — o SNAT que o roteador doméstico faz

**Política padrão `DROP` na cadeia `INPUT` corta a sessão SSH atual no mesmo instante**, se a regra que libera o SSH ainda não estiver no lugar. Em máquina remota a ordem é inegociável: insira `established,related` e o SSH, **confirme numa segunda conexão**, e só então mude a política. Melhor ainda, agende o resgate antes: `echo 'nft flush ruleset' | sudo at now + 5 minutes` — `atrm` cancela se tudo der certo. **E persistir é passo separado, sempre.**

## RESUMO RÁPIDO

- ✓ **Lista ordenada, primeira que casa decide.** Não existe "mais específica vence" — existe posição na fila.
- ✓ **A decisão de rota escolhe o gancho:** para a máquina é `input`, de passagem é `forward`, e um nunca vê o outro.
- ✓ **Com `conntrack` você só decide o que é `new`** — a resposta entra por `established,related`. E `iif lo accept` não é opcional.
- ✓ **`-A` acrescenta ao fim, `-I` insere no topo** — e `-L -n -v --line-numbers` mostra a ordem real e os contadores.
- ✓ **Regra na memória some no reboot:** `nftables.service` ou `iptables-persistent`.

## NO KALI

**O Kali não vem com firewall carregado, de propósito:** `nft list ruleset` numa instalação nova volta vazio e as cadeias do `iptables` estão em `ACCEPT`. Não é falha — uma máquina de teste precisa que o tráfego flua sem obstáculo para não confundir "o firewall bloqueou" com "o alvo não respondeu". A contrapartida é que **a proteção é responsabilidade sua. Firewall de saída é o que contém o shell reverso, e quase ninguém configura:** todo mundo filtra o que entra, poucos filtram o que sai — mas o padrão de um comprometimento é uma conexão de **saída**, iniciada pela máquina invadida, para porta alta num servidor de controle. Uma política `output` restrita transforma isso numa tentativa frustrada e visível no log: melhor custo-benefício, e a mais negligenciada. `conntrack -L` **é análise, não só firewall:** lista o que o kernel rastreia agora — mais difícil de falsear que o `ss`, porque vive no kernel. **Limitar conexão nova contém força bruta na origem** (`limit rate 4/minute`), antes de o `sshd` gastar um ciclo. **Regra escrita não é regra testada:** dispare `nc -zv` e `ping` de um namespace e confira os contadores. **RHEL:** o `firewalld` vem ativo e restritivo, com **zonas** e `firewall-cmd --add-service=https --permanent` — mexer no `nft` por baixo dele gera conflito.